

Apostila Vue.js — Aula 4: Ciclo de Vida e Vite

Oct 5, 2026 · @Someone

Introdução

Nesta aula o projeto ganha estrutura de verdade: saímos do Vue via CDN, criamos o projeto com Vite e passamos a escrever cada componente em seu próprio arquivo `.vue`. Também vamos entender **quando** cada parte de um componente entra em ação (ciclo de vida) e como reagir à mudança de um dado específico (`watch`).

Revisão da Aula 3 — o que você já sabe fazer:

- `app.component()` registra um componente com nome e configuração.
- `props` levam dados do pai para o filho.
- `emits` levam avisos (eventos) do filho para o pai.
- Fluxo de dados: props descem, eventos sobem — nunca o contrário.

1. Por que sair do CDN?

O CDN foi ótimo para aprender sem instalar nada, mas atrapalha quando o projeto cresce. O **Vite** é uma ferramenta de build: organiza, otimiza e serve o projeto Vue durante o desenvolvimento e na hora de publicar.

Aspecto	Com CDN	Com Vite
Organização	Todos os componentes no mesmo <code>app.js</code>	Um arquivo <code>.vue</code> por componente
Pacotes do npm	Não dá para instalar	Instala e importa qualquer pacote
Atualização da tela	Recarregar a página manualmente	Hot reload: atualiza sozinha ao salvar
Produção	Sem passo de build	Um comando gera versão otimizada

Criando o projeto

É preciso ter o **Node.js** instalado. Depois, quatro comandos no terminal:

```
npm create vite@latest minha-app -- --template vue
cd minha-app
npm install
npm run dev
```

- `npm create vite@latest` gera a estrutura inicial do projeto com o template `vue`.
- `npm install` baixa as dependências (incluindo o próprio Vue).
- `npm run dev` sobe um servidor local com hot reload. Abra o endereço mostrado (normalmente `http://localhost:5173/`).

2. Estrutura do projeto

O Vite já cria uma organização padrão. No começo, o que importa mora dentro de `src/`.

```
minha-app/
├── index.html
├── package.json
└── src/
    ├── main.js
    ├── App.vue
    └── components/
        └── TarefaItem.vue
```

Arquivo / pasta	Papel
<code>index.html</code>	Só tem uma <code>div</code> vazia; quem desenha tudo é o Vue
<code>src/main.js</code>	Cria a aplicação e monta na página (o novo lugar do <code>createApp</code>)
<code>src/App.vue</code>	Componente raiz, equivalente ao antigo <code><div id="app"></code>
<code>src/components/</code>	Componentes filhos, um arquivo <code>.vue</code> cada

O `main.js` gerado pelo Vite fica assim:

```
import { createApp } from "vue";
import App from "./App.vue";

createApp(App).mount("#app");
```

3. Single File Components (.vue)

Um arquivo `.vue` reúne em um só lugar o que antes estava espalhado, cada parte na sua seção:

- `<template>` — o HTML do componente.
- `<script>` — a configuração: `data`, `methods`, `props`, `emits`, etc.
- `<style>` — o CSS do componente. Com `scoped`, ele vale só para esse componente.

As três seções são opcionais, mas quase todo componente tem `<template>` e `<script>`. Não é mais preciso `app.component()`: o componente é importado onde for usado (veja o exemplo completo mais abaixo).

Convertendo do CDN para .vue

O mesmo componente `tarefa-item` da Aula 3, antes e depois.

Antes (app.js via CDN):

```
app.component("tarefa-item", {
  props: ["texto"],
  emits: ["marcar-feita"],
  template: `
    <li @click="$emit('marcar-feita')">
      {{ texto }}
    </li>`
});
```

Depois (Tarefaltem.vue):

```
<template>
  <li @click="$emit('marcar-feita')">
    {{ texto }}
  </li>
</template>

<script>
export default {
  props: ["texto"],
  emits: ["marcar-feita"]
};
</script>
```

A lógica é a mesma. Muda apenas onde cada parte mora: o HTML vai para `<template>`, e a configuração vira o objeto exportado (`export default`) em `<script>`.

4. Ciclo de vida

Todo componente passa por fases entre nascer e sair da tela, e o Vue avisa cada fase com um **hook**. Um hook é só um método com nome especial: o Vue o chama sozinho, no momento certo.

As fases, na ordem em que acontecem:

1. `created` — os dados já existem, mas a tela ainda não foi desenhada.
2. `mounted` — o componente já está na tela (DOM pronto).
3. `updated` — um dado mudou e a tela foi atualizada (pode acontecer várias vezes).
4. `unmounted` — o componente saiu da tela.

`created` e `mounted` são os mais usados no dia a dia. `updated` e `unmounted` existem, mas aparecem menos no início.

`created()` x `mounted()`

Critério	<code>created()</code>	<code>mounted()</code>
A tela (DOM) já existe?	Ainda não	Sim
Roda antes ou depois de desenhar?	Antes	Depois
Bom para...	Buscar dados iniciais, preparar valores	Medir elementos, integrar bibliotecas de terceiros

Regra prática: se só precisa preparar dados, use `created()`. Se precisa tocar em algo que existe na tela (um elemento, um gráfico), use `mounted()`.

```
export default {
  data() {
    return { mensagem: "" };
  },
  created() {
    // dados prontos, tela ainda não
    this.mensagem = "Olá!";
  },
  mounted() {
    // tela pronta: posso acessar o DOM
    console.log(this.$el);
  }
};
```

Repare que os hooks ficam **direto no objeto exportado**, ao lado de `data` — não dentro de `methods`.

5. watch: reagindo a mudanças de um dado

`watch` executa uma função sempre que um dado específico muda. É ideal para **efeitos colaterais**: buscar dados, salvar algo, registrar um log. A função tem o mesmo nome do dado observado e recebe dois argumentos: o valor novo e o valor antigo.

```
export default {
  props: ["filtro"],
  data() {
    return { resultados: [] };
  },
  watch: {
    filtro(novoValor, valorAntigo) {
      console.log(`mudou de ${valorAntigo} para ${novoValor}`);
      this.buscarResultados();
    }
  },
  methods: {
    buscarResultados() {
      // busca os dados de acordo com this.filtro
    }
  }
};
```

watch x computed

	computed	watch
Devolve um valor?	Sim, para exibir na tela	Não
Para que serve	Calcular um valor a partir de outros dados	Executar uma ação quando um dado muda
Exemplo	<code>total</code> de tarefas concluídas	Salvar a lista sempre que ela mudar

6. Exemplo completo

Juntando tudo: um `TarefaItem.vue` com props, emits, `created`, `mounted` e `watch`, usado dentro do `App.vue`.

src/components/TarefaItem.vue

```

<template>
  <li @click="$emit('marcar-feita')" :class="{ feita: feita }">
    {{ texto }}
  </li>
</template>

<script>
export default {
  props: ["texto", "feita"],
  emits: ["marcar-feita"],
  created() {
    console.log("tarefa criada:", this.texto);
  },
  mounted() {
    console.log("tarefa na tela");
  },
  watch: {
    feita(novoValor) {
      console.log("feita agora é:", novoValor);
    }
  }
};
</script>

<style scoped>
.feita {
  text-decoration: line-through;
  color: gray;
}
</style>

```

src/App.vue

```

<template>
  <ul>
    <TarefaItem
      v-for="(tarefa, i) in tarefas"
      :key="i"
      :texto="tarefa.texto"
      :feita="tarefa.feita"
      @marcar-feita="tarefa.feita = !tarefa.feita"
    />
  </ul>
</template>

```

```
<script>
import TarefaItem from "../components/TarefaItem.vue";

export default {
  components: { TarefaItem },
  data() {
    return {
      tarefas: [
        { texto: "Estudar Vite", feita: false },
        { texto: "Revisar props", feita: true }
      ]
    };
  }
};
</script>
```

Abra o console do navegador (F12): ao carregar, aparecem os logs de `created` e `mounted` de cada tarefa. Ao clicar em uma tarefa, o `watch` registra o novo valor de `feita`.

Resumo

Conceito	O que é
Vite	Ferramenta de build: organiza o projeto e roda um servidor com hot reload
Single File Component	Um arquivo <code>.vue</code> que reúne template, script e style
<code>created()</code>	Roda quando os dados já existem, antes de desenhar a tela
<code>mounted()</code>	Roda quando o componente já está na tela
<code>watch</code>	Executa uma ação sempre que um dado observado muda

Próxima aula: Vue Router — navegação entre páginas.

Exercícios

Os exercícios vão do mais simples ao mais completo. Os de 1 a 4 são de fixação; os de 5 a 10 são práticos e pedem o projeto Vite rodando.

1. Verdadeiro ou falso. Justifique cada resposta.

- a) No CDN, a tela atualiza sozinha sempre que salvamos o arquivo.
- b) `npm install` baixa as dependências do projeto, incluindo o Vue.
- c) Em um arquivo `.vue`, a seção `<style>` é obrigatória.
- d) No `created()`, já é possível acessar os elementos da tela.

2. Estrutura do projeto. Explique com suas palavras a função de `index.html`, `main.js`, `App.vue` e da pasta `components/`.**3. created ou mounted?** Para cada situação, diga qual hook usar e por quê:

- a) Definir o valor inicial de uma variável a partir de uma prop.
- b) Medir a largura de um elemento da tela.
- c) Iniciar um gráfico de uma biblioteca externa dentro de uma `div`.
- d) Buscar uma lista inicial de produtos.

4. watch ou computed? Para cada situação, diga qual usar:

- a) Mostrar o nome completo a partir de `nome` e `sobrenome`.
- b) Registrar no console toda vez que o `email` mudar.
- c) Exibir quantas tarefas ainda faltam.
- d) Salvar a lista de tarefas sempre que ela for alterada.

5. Primeiro projeto. Crie um projeto chamado `aula4-exercicios` com o Vite. Rode `npm run dev`, abra no navegador e troque o conteúdo do `App.vue` por um `<h1>` com seu nome. Observe o hot reload ao salvar.**6. Convertendo do CDN.** Transforme o componente abaixo, feito para CDN, em um arquivo `Saudacao.vue` e use-o no `App.vue`:

```
app.component("saudacao", {
  props: ["nome"],
  template: `<p>Olá, {{ nome }}!</p>`
});
```

7. Estilo só do componente. No `Saudacao.vue`, adicione um `<style scoped>` que deixe o parágrafo azul. Coloque também um `<p>` direto no `App.vue` e confirme que ele **não** ficou azul.**8. Observando o ciclo de vida.** Crie um componente `Contador.vue` com um dado `numero` e um botão que soma 1. Coloque um `console.log` em `created`, `mounted` e `updated`. Abra o console e anote em que ordem cada mensagem aparece e quais se repetem ao clicar.

9. Watch com valor antigo e novo. Crie um componente `Busca.vue` com um `<input v-model="termo">`. Use `watch` em `termo` para mostrar no console: "busca mudou de X para Y". Desafio: se o termo tiver menos de 3 letras, mostre "termo muito curto" em vez da mensagem.

10. Projeto final: lista de tarefas. Monte uma lista de tarefas com `App.vue` e `TarefaItem.vue`, seguindo os requisitos:

- ☐ `TarefaItem.vue` recebe `texto` e `feita` como props.
- ☐ No `created()`, loga "tarefa criada:" com o texto; no `mounted()`, loga "tarefa na tela".
- ☐ Ao clicar, emite `marcar-feita`, e o `App.vue` inverte o valor de `feita`.
- ☐ Um `watch` na prop `feita` loga o novo valor.
- ☐ Tarefas feitas aparecem riscadas (`<style scoped>`).
- ☐ Extra: um campo no `App.vue` para adicionar novas tarefas e um `computed` que mostra quantas faltam.